

Kubernetes and Cloud Native Associate - KCNA

Cloud Native Architecture

Vertical Pod Autoscaler

In this lesson, we will explore the Vertical Pod Autoscaler (VPA) and its role in dynamically adjusting resource allocations for Kubernetes pods. We'll start with the basics of resource requirements in pods using a simple web application example, and then dive into how the VPA components interact to manage and optimize these resources automatically.

Resource Requirements for Pods

A fundamental aspect of Kubernetes is the proper allocation of resources to pods. In a pod specification, you define two critical parameters:

- **Resource Requests:** Guarantee that the container will receive a specific amount of resources. For instance, if a container requests 64 megabytes of memory and 250 CPU units (i.e., 250m), Kubernetes schedules it only on nodes that can provide these resources.
- **Resource Limits:** Ensure that a container does not use more resources than allowed. In our example, the container has a limit of 128 megabytes of memory and 500 milli CPU units, preventing it from exceeding this threshold.

Below is an example pod specification with resource allocation defined:

```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
    - name: simple-webapp-color
      image: simple-webapp-color
      ports:
        - containerPort: 8080
      resources:
        requests:
          memory: "64Mi"
          cpu: 250m
        limits:
          memory: "128Mi"
          cpu: 500m
```

Kubernetes enforces these definitions while the pod is running. However, under increased traffic or load, the preset resource allocation might be insufficient. When usage approaches or exceeds limits, pods may become unstable or even crash due to resource starvation.

Introduction to the Vertical Pod Autoscaler

The Vertical Pod Autoscaler (VPA) is designed to adjust CPU and memory resources for pods based on real-time workload demands. It continuously monitors resource utilization and recommends or enacts changes to maintain optimal performance.

Imagine a scenario where the VPA's recommender component notices that a pod is frequently pushing its resource limits—such as using 190Mi out of 192Mi of memory or 700m out of 750m CPU. In such cases, the recommender might suggest

increasing both the memory and CPU allocations to ensure stability and performance.

VPA Components Explained

1. **Recommender:**

The recommender monitors resource usage and provides suggestions to adjust requests and limits based on observed behavior.

2. **Updater:**

The updater compares the current pod configuration with the recommended settings. If differences are detected, it initiates actions (such as evicting the pod) to apply the new resource allocations. Note that Kubernetes traditionally does not allow the direct modification of running pods' resource requests. Starting with Kubernetes version 1.28, in-place updates of pod resources reduce or eliminate the need for evictions.

3. **Admission Controller:**

This component intercepts new pod creation requests. When a pod is restarted or created (for example, after an eviction), the admission controller updates the pod's resource configuration based on the recommender's suggestions.

Update Policy Modes

VPA supports different update policy modes to control how resource recommendations are applied:

- **Off:**

The VPA provides recommendations but does not automatically update the pod's resource requests. Manual intervention is required to apply any suggested changes.

- **Initial:**

Automatically sets optimal resource requests at pod creation but does not modify running pods.

- **Auto:**
Automatically applies recommended resource adjustments both during pod creation and throughout the pod's lifecycle. In Kubernetes versions prior to 1.28, this mode required pod evictions to apply changes. Although more aggressive, the Auto mode ensures that pods run with the optimal configuration based on usage patterns.

For instance, a resource block in a pod specification may appear as follows:

```
resources:
  requests:
    memory: "64Mi"
    cpu: 250m
  limits:
    memory: "128Mi"
    cpu: 500m
```

Deploying the VPA Components

The VPA is not enabled by default in Kubernetes clusters. To deploy the VPA components, follow these steps:

1. Clone the Kubernetes autoscaler repository.
2. Navigate to the vertical pod autoscaler directory.
3. Execute the provided hack script to deploy the necessary components.

```
git clone https://github.com/kubernetes/autoscaler.git
cd autoscaler/vertical-pod-autoscaler/
./hack/vpa-up.sh
```

This script deploys VPA components as Deployments, sets up the required RBAC (Role-Based Access Control) configurations, and creates other necessary objects

for proper cluster operation.

Configuring a Vertical Pod Autoscaler Object

After deploying the VPA components, the next step is to create a VPA object that references your target pod and defines its resource parameters. In this example, a VPA is configured for a pod named "simple-webapp-color" with the update mode set to "Off", meaning the VPA will only provide recommendations without automatically modifying the pod.

First, here is the pod specification for the web application:

```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
    - name: simple-webapp-color
      image: simple-webapp-color
      ports:
        - containerPort: 8080
  resources:
    requests:
      memory: "64Mi"
      cpu: 250m
    limits:
      memory: "128Mi"
      cpu: 500m
```

Then, create the corresponding VPA object:

```
apiVersion: "autoscaling.k8s.io/v1"
kind: VerticalPodAutoscaler
metadata:
  name: webapp-vpa
spec:
  targetRef:
    apiVersion: "v1"
    kind: Pod
    name: simple-webapp-color
  updatePolicy:
    updateMode: "Off"
```

Once the VPA object is up and running, view the recommendations with the following command:

```
kubectl describe vpa webapp-vpa
```

The output details recommended CPU and memory settings along with defined lower and upper bounds. For example:

```
Name:          webapp-vpa
Namespace:     default
Labels:        <none>
Annotations:   <none>
API Version:   autoscaling.k8s.io/v1
Kind:          VerticalPodAutoscaler
...
Status:
  Recommendation:
    Container Recommendations:
      Container Name:  webapp
      Lower Bound:
        Cpu:          500m
        Memory:       128Mi
      Target:
        Cpu:           1
        Memory:       256Mi
```

Uncapped Target:

Cpu: 1
Memory: 256Mi

Upper Bound:

Cpu: 2
Memory: 512Mi

...

The "Target" values represent the current recommendations, while "Lower Bound" and "Upper Bound" provide the feasible range based on observed resource usage. Adjust these recommendations as needed for your production environment.



Note

For information on working with Kubernetes resource allocations and autoscaling, refer to [Kubernetes Basics](#).

Conclusion

This article provided an in-depth overview of the Vertical Pod Autoscaler, detailing how it works in tandem with resource requests and limits to keep Kubernetes pods optimally configured. By understanding the interaction between VPA components and how they dynamically adjust resource allocations, you can better manage workloads and ensure consistent application performance even as demand fluctuates.

Thank you for reading, and see you in the next lesson!



Watch Video

Watch video content

Previous

◀ Horizontal Pod Autoscaler

Next

Cluster Autoscaler ▶



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ▼